Structure du Document :

1. **Architecture Globale (rosetta.h) :** les structures (struct), le mot-clé typedef, et la règle de manipulation des pointeurs (-> vs .). Analyse pas à pas de l'utilité des structures t_parse, t_buf, t_entry (liste chaînée) et t_dict.
2. **Validation des Entrées (check_input.c) :** Décryptage de la sécurisation des arguments, de la détection et du rejet immédiat des nombres négatifs, et de l'ajustement du pointeur pour éliminer proprement les zéros non significatifs à gauche.
3. **Gestion & Extraction du Dictionnaire (load_dictionary.c & parsing_utils.c) :** Analyse de la lecture octet par octet via read(), de la détection des sauts de lignes et du basculement binaire (mode 0 pour la clé numérique / mode 1 pour le mot littéral).
4. **Allocation Dynamique & Stockage (dict_utils.c) :** Cours théorique sur le fonctionnement du **Tas** (*Heap*), de la protection obligatoire des retours de malloc et du calcul des puissances de triplets (compteur de triplets de zéros).
5. **Algorithmes d'Exploration (lookup_exception.c) :** Analyse de la recherche séquentielle des exceptions directes et des échelles. Contient le cours de nettoyage de liste chaînée : la sauvegarde impérative de l'adresse suivante (next_node = curr->next) avant d'exécuter un free(curr) pour interdire les fuites.
6. **Gestion des Tampons de Sortie (print_dict_word.c) :** Explication de la copie sécurisée des caractères depuis l'arène de stockage fixe vers le buffer de sortie global en évitant les débordements de tampons.
7. **Décomposition Algorithmique des Blocs (print_triplet_value.c) :** Logique d'analyse des nombres de 0 à 999. Elle documente l'ajustement mathématique automatique pour la langue française lors de l'interception des dizaines complexes (70 et 90).
8. **Algorithme Maître de Découpage (print_large_number.c) :** Explication de la formule de calcul du décalage (Modulo initial total_len % 3) et de la formule de résolution des échelles de grandeurs restantes.
9. **Régulation et Compilation (main.c & Makefile) :** Rôle des drapeaux stricts (-Wall -Wextra -Werror), du drapeau de débogage pour Valgrind (-g3), des règles de substitution implicites %.o: %.c et de la directive virtuelle .PHONY.
10. **Banc de Validation & Mémoire (test_matrix.sh & Valgrind) :** Cours de scripting Bash (Shebang #!/bin/bash, statut $?), redirection avancée des flux d'erreurs (2>&1), assignation de code d'arrêt forcé (--error-exitcode=42), et nettoyage des logs avec grep -v '==' .
11. **Trace Complète sur Exemple (Nombre : 1245684) :** Guide pas à pas décrivant l'état des registres, du buffer et des variables à chaque transition de bloc ("1", puis "245", puis "684").
12. **Moniteur d'Interception des Erreurs :** Schéma explicatif complet différenciant le circuit d'erreur de paramètres ("Error\n") du circuit de corruption ou de manque de mémoire ("Dict Error\n").

1. ARCHITECTURE GLOBALE (`rosetta.h`)
Les Structures (`struct`) et Pointeurs en C

Une structure (`struct`) regroupe des variables de types différents sous un seul nom en mémoire.

- **Passage par variable :** On accède aux champs avec le point (`.`). Exemple : `structure.champ = valeur;`.
- **Passage par pointeur :** On accède aux champs avec la flèche (`->`). Exemple : `pointeur->champ = valeur;`. C'est l'équivalent raccourci de `(*pointeur).champ`.
- **`typedef` :** Permet de créer un alias pour éviter d'écrire le mot-clé `struct` à chaque déclaration.
- Syntax :
    - **`typedef` (Type Definition) :** mot-clé du langage C qui sert à créer un *alias* (un raccourci de nom) pour un type existant.
    - **`struct` :** C'est le mot-clé qui indique au compilateur que vous allez regrouper plusieurs variables dans un même bloc de mémoire.
    - **`s_entry` (Le "Tag" de la structure) :** C'est le nom de la structure d'origine. Par convention de l'école 42 il commence par **`s_`** (“structure”)
    - **`{ ... }` (Le corps de la structure) :** C'est l'espace où vous déclarez les variables internes, aussi appelées les "membres" ou les "champs" de la structure.
    - **`t_entry` (L'alias de type) :** C'est le nom du raccourci final. Par convention, il commence par **`t_`** (pour "type").

```
typedef struct s_parse {
    int mode;       // 0 = lecture de la clé numérique, 1 = lecture du mot
    int n_idx;      // Index actuel dans le tampon de la clé numérique
    int t_idx;      // Index actuel dans le tampon du mot textuel
    int has_colon;  // Drapeau indiquant si le symbole ':' a été trouvé
} t_parse;

typedef struct s_buf {
    char str[16384]; // Tampon de chaîne fixe pour accumuler le texte final
    int len;         // Longueur active actuelle de la chaîne stockée
} t_buf;

typedef struct s_entry {
    int value;            // Valeur numérique (0-999) ou -1 pour une échelle
    int triplet_count;    // Nombre de triplets (ex: 1 = mille, 2 = million)
    int text_offset;      // Position de départ du mot dans la zone de texte
    struct s_entry *next; // Pointeur de référence vers le nœud suivant
} t_entry;

typedef struct s_dict {
  t_entry *head;        // Premier nœud de la liste chaînée du dictionnaire
  t_buf out_buf;        // Instance du tampon d'affichage global
  char text_arena[16384]; // Zone mémoire fixe où sont stockés tous les mots
  int arena_len;        // Nombre d'octets actuellement occupés dans l'arène
  int error_state;      // Indicateur d'erreur (0 = OK, 1 = Erreur critique)
} t_dict;
```

2. VALIDATION DES ENTRÉES (`check_input.c`)

```
static int validate_string_number(const char *str, int *start, int *end)
{
    int i = 0;
    int has_digit = 0;

    if (str[i] == '-')         // Interception directe des nombres négatifs
        return (0);            // Invalide : renvoie faux immédiatement
    if (str[i] == '+')         // Ignore le signe positif initial
        i++;
    *start = i;                // Enregistre l'index du début réel du nombre
    while (str[i] >= '0' && str[i] <= '9') // Parcourt que les chiffres
    {
        has_digit = 1;
        i++;
    }
    if (!has_digit)            // Si aucun chiffre après le signe
        return (0);
    *end = i;                  // Enregistre la fin
    return (1);                // Validation réussie
}

int check_input(int argc, char **argv, const char **path, char **val_str)
{
    int start = 0;
    int end = 0;

    *path = "numbers.dict";    // Dictionnaire par défaut
    if (argc < 2 || argc > 3)  // Nombre incorrect d'arguments en entrée
        return (0);
    *val_str = argv[argc - 1]; // Le nombre à traiter est toujours le
dernier argument
    if (argc == 3)
        *path = argv[1];       // Dictionnaire personnalisé en paramètre
    if (!validate_string_number(*val_str, &start, &end))
        return (0);
    *val_str = &((*val_str)[start]); // Ajuste pointeur si un '+'
    (*val_str)[end - start] = '\0';  // Tronque & ferme la chaîne
    return (1);
}
```

3. PARSING DU DICTIONNAIRE (`load_dictionary.c` & `parsing_utils.c`)

```
void parse_char(t_dict *d, char c, char *buf, t_parse *p)
{
    if (p->mode == 0)            // Mode lecture du nombre de gauche
    {
        if (c == ':')
        {
            buf[p->n_idx] = '\0'; // Ferme la chaîne contenant le nombre
            p->mode = 1;          // Bascule en mode lecture du texte
            p->has_colon = 1;
        }
        else if (c >= '0' && c <= '9' && p->n_idx < 120)
            buf[p->n_idx++] = c;   // Accumule le caractère numérique
    }
  else if (p->mode == 1 && d->arena_len + p->t_idx < 16384 - 2)
                                   // Mode texte
    {
        if (p->t_idx > 0 || (c != ' ' && c != '\t'))
                                   // Ignore les espaces initiaux
        {
            d->text_arena[d->arena_len + p->t_idx] = c;
                                   // Écrit dans l'arène centrale
            p->t_idx++;
        }
    }
}

int reset_or_save(t_dict *d, char *num_buf, t_parse *p)
{
    if (p->n_idx > 0 || p->t_idx > 0)
    {
        if (!p->has_colon) // Ligne corrompue sans séparateur ':'
        {
            d->error_state = 1;
            return (-1);
        }
        if (p->mode == 1 && p->t_idx > 0)
            save_entry(d, num_buf, p->n_idx, p->t_idx);
                        // Sauvegarde l'élément
    }
    p->mode = 0; // Réinitialisation complète pour traiter ligne suivante
    p->n_idx = 0;
    p->t_idx = 0;
    p->has_colon = 0;
    return (1);
}
```

4. ALLOCATION ET CRÉATION DE LA LISTE (`dict_utils.c`)

L'Allocation Dynamique et le Tas

Contrairement à la **Pile** (*Stack*) où les variables locales sont détruites automatiquement à la fin d'une fonction, le **Tas** (*Heap*) permet de réserver de la mémoire persistante via `malloc()`.

- **Règle de sécurité :** Tout retour de `malloc` doit être testé. Si `malloc` renvoie `NULL`, le système manque de mémoire.

```
static void assign_node_scales(t_entry *new_node, char *nb_buf, int n_idx)
{
    int val = 0;
    int i = -1;

    while (++i < n_idx && n_idx <= 3)
        val = (val * 10) + (nb_buf[i] - '0'); // Conversion de chaîne vers entier
    new_node->value = val;
    new_node->triplet_count = 0; // C'est un nombre standard de 0 à 999

    if (n_idx >= 4 && nb_buf[0] == '1') // Détection d'une échelle (1000, 1000000,
etc.)
    {
        new_node->value = -1; // Marqueur d'échelle
        new_node->triplet_count = (n_idx - 1) / 3; // Calcule la puissance de 1000
    }
}

void save_entry(t_dict *d, char *nb_buf, int n_idx, int t_idx)
{
    t_entry *new_node;

    if (d->arena_len + t_idx + 1 >= 16384) // Sécurité anti-débordement de l'arène
    {
        d->error_state = 1;
        return;
    }
    new_node = malloc(sizeof(t_entry)); // Allocation sur le Tas
    if (!new_node)
    {
        d->error_state = 1;
        return;
    }
    d->text_arena[d->arena_len + t_idx] = '\0'; // Termine le mot dans l'arène
    new_node->text_offset = d->arena_len;       // Enregistre sa position d'accès
    d->arena_len += t_idx + 1;                  // Avance le compteur de l'arène
    nb_buf[n_idx] = '\0';
    assign_node_scales(new_node, nb_buf, n_idx);

    new_node->next = d->head; // Insertion en tête de liste chaînée
    d->head = new_node;
}
```

5. RECHERCHE ET LIBÉRATION SÉCURISÉE (`lookup_exception.c`)

Destruction de Liste et Pointers Suspendus

Quand on libère une liste chaînée avec `free()`, il ne faut jamais détruire un nœud avant d'avoir lu l'adresse du nœud suivant, sous peine de perdre le reste de la liste (*Memory Leak*) ou de provoquer un crash.

```
t_entry *lookup_exception(t_dict *dict, int value)
{
    t_entry *curr = dict->head;
    while (curr != NULL)
    {
        if (curr->value == value && curr->triplet_count == 0)
            return (curr); // Renvoie l'élément trouvé (ex: 15 -> fifteen)
        curr = curr->next;
    }
    return (NULL);
}

void free_dictionary(t_dict *dict)
{
    t_entry *curr;
    t_entry *next_node;

    if (!dict || !dict->head)
        return;
    curr = dict->head;
    while (curr != NULL)
    {
        next_node = curr->next; // 1. Sauvegarde le pointeur suivant
impérativement !
        free(curr);             // 2. Libère le nœud actuel sur le tas
        curr = next_node;       // 3. Passe au nœud sauvegardé sans risquer
de dangle pointer
    }
    dict->head = NULL;
}
```

6. GESTION DU TAMPON DE SORTIE (`print_dict_word.c`)

```
void print_dict_word(t_dict *dict, t_entry *node)
{
    int i = 0;
    int offset;

    if (!node)
        return;
    offset = node->text_offset;
    while (dict->text_arena[offset + i] != '\0') // Extrait le mot de
l'arène
    {
        if (dict->out_buf.len < 16384 - 1) // Sécurité d'écriture dans le
tampon global
        {
            dict->out_buf.str[dict->out_buf.len] = dict->text_arena[offset +
i];
            dict->out_buf.len++;
        }
        i++;
    }
    dict->out_buf.str[dict->out_buf.len] = '\0'; // Assure la fermeture de
la chaîne
}
```

7. LOGIQUE DES BLOCS DE 3 CHIFFRES (`print_triplet_value.c`)

```
static int print_tens_units(t_dict *dict, int tens, int units)
{
    t_entry *node;

    node = lookup_exception(dict, tens);
    if (!node)
    {
        if (tens == 70 || tens == 90) // Interception et adaptation de la
grammaire française
        {
            tens -= 10; // Devient 60 ou 80
            units += 10; // Devient 11-19 ou 10-19
        }
    }
    node = lookup_exception(dict, tens);
    if (!node)
        return (-1);
    print_dict_word(dict, node);
    if (units > 0)
    {
        if (dict->out_buf.len < 16384 - 1)
            dict->out_buf.str[dict->out_buf.len++] = '-'; // Ajout du tiret
syntaxique
        append_unit(dict, units);
    }
    return (0);
}

void print_triplet_value(t_dict *dict, int n)
{
    t_entry *node;

    if (n >= 100) // Traitement des centaines
    {
        handle_hundreds(dict, &n);
    }
    if (n > 0)
    {
        node = lookup_exception(dict, n); // Vérifie s'il existe une
exception (0-19, dizaines pures)
        if (node)
            print_dict_word(dict, node);
        else
        {
            if (print_tens_units(dict, (n / 10) * 10, n % 10) == -1) //
Découpage dizaine-unité
                dict->error_state = 1;
        }
```

```
    }
}
```

8. ALGORITHME MAÎTRE DE DÉCOUPE (`print_large_number.c`)

```
static int split_and_print(t_dict *dict, const char *s, int total_len)
{
    int pb = 0;
    int rem = total_len % 3; // Calcul du reste (Modulo de décalage initial)
    int idx = 0;
    int c_len;
    int triplets_left;

    while (idx < total_len)
    {
        c_len = 3;
        if (idx == 0 && rem > 0)
            c_len = rem; // Ajuste la taille du tout premier bloc si
incomplet

        // Formule maîtresse de résolution des échelles de grandeurs
restantes
        triplets_left = (total_len - (idx + c_len)) / 3;

        if (process_chunk(dict, get_chunk_val(s, idx + c_len, c_len),
triplets_left, &pb) == -1)
            return (-1);
        idx += c_len; // Avance l'index dans la chaîne globale
    }
    return (0);
```

9. ORCHESTRATION ET AUTOMATISATION (`main.c` & `Makefile`)

Rappel sur le Makefile

- **`CFLAGS` :** `-Wall -Wextra -Werror` force le compilateur à être intransigeant. Tout avertissement (warning) bloque la compilation.
- **`-g3` :** Injecte les symboles de débogage avancés nécessaires pour que Valgrind affiche le numéro exact des lignes de code fautives.
- **`.PHONY` :** Indique à `make` que les règles (`clean`, `fclean`, `re`) sont des commandes et non des fichiers physiques à créer.

Analyse du Code Source (`main.c`)

c

```
int main(int argc, char **argv)
{
    t_dict dict;
    const char *dict_path;
    char *num_str;

    dict.head = NULL;
    if (!check_input(argc, argv, &dict_path, &num_str)) // Route Erreur 1
    {
        write(2, "Error\n", 6); // Sortie d'erreur 2 (standard)
        return (1);
    }
    if (load_dictionary_from_file(&dict, dict_path) <= 0) // Route Erreur 2
    {
        write(2, "Dict Error\n", 11);
        return (1);
    }
    print_large_number(&dict, num_str); // Traitement applicatif
    free_dictionary(&dict);              // Nettoyage absolu du Tas
    return (0);
}
```

10. VALIDATION AUTOMATIQUE ET MÉMOIRE (`test_matrix.sh` & Valgrind)

Scripting Shell et Interception de Flux

- **`2>&1` :** Fusionne l'erreur standard (canal 2) avec la sortie standard (canal 1) pour capturer l'ensemble des réponses dans une seule variable.
- **`$?` :** Variable système stockant le statut d'arrêt de la dernière commande exécutée (0 = Succès, différent de 0 = Échec).

bash

```
# Lancement de l'exécutable sous Valgrind avec code d'erreur dédié 42 en cas de fuite mémoire
raw_output=$(valgrind --leak-check=full --error-exitcode=42 $EXEC "$dict_file" "$input_val" 2>&1)
exit_status=$?

if [ $exit_status -eq 42 ]; then
    echo -e "Result: FAIL (Fuite mémoire ou crash détecté par Valgrind !)"
    exit 1
fi

# Nettoyage des lignes de logs Valgrind (qui commencent par ==) pour comparer uniquement le texte
out=$(echo "$raw_output" | grep -v '==' | xargs)
```

Use code with caution.

11. TRACE D'ÉXÉCUTION COMPLÈTE (EXEMPLE : 1245684)

1. **`main.c` / `check_input.c` :** Longueur de la chaîne `"1245684"` = 7. Calcul du reste initial : `7 % 3 = 1`. Taille du premier bloc = 1.
2. **Traitement du Bloc 1 :** Chaîne `"1"` → valeur entière = `1`.

- `triplets_left` = (7 - (0 + 1)) / 3 = 2.
- `lookup_exception(1)` → **"one"**.
- `lookup_scale(2)` → **"million"**.
- `out_buf` contient : `one million`.

3. **Traitement du Bloc 2 :** Chaîne `"245"` → valeur entière = `245`.

- `triplets_left` = $(7 - (1 + 3)) / 3 = 1`.
- `print_triplet_value` décompose 245 $\rightarrow$ **"two hundred forty-five"**.
- `lookup_scale(1)` $\rightarrow$ **"thousand"**.
- `out_buf` contient : `one million two hundred forty-five thousand`.

4. **Traitement du Bloc 3 :** Chaîne `"684"` $\rightarrow$ valeur entière = `684`.

- `triplets_left` = $(7 - (4 + 3)) / 3 = 0`.
- `print_triplet_value` décompose 684 → **"six hundred eighty-four"**.
- `triplets_left == 0` → Aucune échelle ajoutée.

5. **Sortie Écran :** L'application appelle `write(1, out_buf.str, out_buf.len)` et libère proprement la mémoire.

Text en sortie :

```
one million two hundred forty-five thousand six hundred eighty-four
```
Use code with caution.

12. ARCHITECTURE DE GESTION ET CIRCUITS D'ERREURS

```
       [ Lancement du Programme ]
                   │
                   ▼
      Vérification des paramètres?
        ├─► NON (ex: argc incorrect, lettres) ──► Affiche "Error\n" (Canal 2) ──► Exit(1)
        └─► OUI
                   │
                   ▼
     Ouverture et Parsing du dictionnaire?
        ├─► NON (ex: fichier introuvable, colon manquant) ──► Affiche "Dict Error\n"
(Canal 2) ──► Libère le Tas ──► Exit(1)
        └─► OUI
                   │
                   ▼
     Exécution nominale de l'algorithme
        ├─► Découpage par blocs de 3 chiffres de gauche à droite
        ├─► Extraction des exceptions et mots d'échelles associés
        └─► Écriture sécurisée dans le tampon de sortie global (out_buf)
                   │
                   ▼
     [ Affichage du résultat sur la sortie standard ] ──► Libération complète de la
mémoire via free_dictionary() ──► Exit(0)
```
Use code with caution.